

A PARALLEL IMPLEMENTATION OF MONTE CARLO LOCALIZATION FOR MOBILE ROBOTICS

Mugdha Meda
ME21B119

Sumedha Prabha T
ME21B201

Guide
Prof. Kameswararao Anupindi

ABSTRACT

Monte Carlo Localization (MCL) is a robust particle filter-based algorithm that is widely used to locate robots within known environments. It estimates the robot's position and orientation as it moves and senses its surroundings. The algorithm uses probabilistic approaches that allow it to handle uncertainty effectively, which sets it apart from traditional methods. By representing the robot's pose as a probability distribution over the state space, MCL can account for sensor noise, environmental variability, and other sources of uncertainty leading to reliable localization results.

With the introduction of multi-core processing capabilities in today's electronic devices, there is much incentive to parallelize the localization process and make efficient use of computational resources. Applications include indoor navigation, mobile bots, autonomous vehicles (in the absence of GPS signals) and many others. Our project aims to implement MCL, for autonomous navigation of a robot placed anywhere within a predefined 2D map. Parallelization paradigms such as OpenMP and OpenACC will be utilized. Our approach aims to minimize the time taken for localization while mitigating overheads and communication expenses associated with parallelization.

1. INTRODUCTION

Monte Carlo Localization (MCL), also known as Particle Filter Localization, is a powerful technique for helping robots determine their position and orientation in com-

plex and uncertain environments. This method uses a set of "particles," each representing a possible state for the robot. As the robot moves and gathers sensor data, these particles are updated and refined, eventually converging to a likely pose. This method incorporates noise using random generator functions, simulating sensor noise and environmental variability. This makes MCL a popular choice for robotic applications, from warehouse automation to autonomous vehicles.

In this project we explored parallel paradigms like OpenMP and OpenACC to gain a speedup over MCL's serial implementation. The MCL algorithm by nature has great potential for parallelization. This fact is strongly supported by the results of our implementation.

2. ENVIRONMENT AND SETUP

Our implementation of Monte Carlo Localization involved an initial setup of a pre-defined 2D map, the robot's pose, sensors involved and evaluation of cost with noise simulated.

1. 2-D World map
2. Robot's pose
3. Sensor
4. Evaluation of cost

2.1 2-D World Map

Our static environment works with a 100 X 100 world size, with 16 landmarks placed on the map. The landmarks

are simulations of potential obstacles in the real world and the robot is forbidden to move across/over them.

2.2 Robot's Pose

For simplicity, the bot is considered to be a point object. The pose of our bot is defined by its coordinates and orientation (CCW angle from the horizontal.)

$$\text{Pose} = \{x, y, \theta\}$$

2.3 Sensor

We have considered a LiDAR with a 360° field of view and a large enough range to sense all obstacles to maintain simplicity. A practical field of view would be less than 180° and a limited distance up to which the robot can detect.

2.4 Evaluation of Cost

The cost of a particle or the bot is taken in terms of the distances between the particle/bot and the landmarks(obstacles). The cost of the particles and the bot are compared to assign weights to the particles.

3. MONTE CARLO LOCALIZATION

The implementation of the Monte Carlo algorithm for localization is summarized in the following four steps:

1. Particle Generation
2. Motion update
3. Sensor update and Weight assignment
4. Resampling

3.1 Particle Generation

The localization begins with the robot being spawned randomly on the given map. The bot then spawns a given number of uniform, continuously distributed particles over the map in the first iteration. In the subsequent iterations, the particles are generated close to those particles which were assigned greater weights in the previous iteration.

3.2 Motion Update

The robot is given a command to move a particular distance and angle after the generation of new particles. This motion is then mimicked by the particles with a certain

amount of noise to account for uncertainty in actual motion. The particle distribution evolves as the robot moves through the environment. It is to be noted that if the bot encounters a boundary/wall upon acting on the motion command, it will make a 180° turn and continue with its command.

3.3 Sensor update and Weight assignment

Once the particles move, we obtain the cost associated with the LiDAR measurement from the robot and compare it with the simulated LiDAR measurement of the particles. Weights are assigned to each particle based on how well the particle's state aligns with the robot's observed position. This is done using a Gaussian function to evaluate the likelihood and assign weights:

$$\text{Gauss}(\mu, \sigma, x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

X_t is the belief at a time t represented by a set of M particles and $W = w_j : j = 1 \dots M$ is the set of weights of all particles.

The algorithm for generating weights is as follows:

```

 $X_{t-1} = \{x_{t-1}^{[1]}, x_{t-1}^{[2]}, \dots, x_{t-1}^{[M]}\}$ 
 $X_t = \text{MotionUpdate}()$ 
for  $j$  in range  $(1, M)$ 
{
    for  $i$  in range  $(0, \text{Number of Landmarks})$ 
    {
         $M_t = \text{Measurement}(\text{Bot}, \text{Landmark}_i)$ 
         $m_t = \text{Measurement}(x_i, \text{Landmark}_i)$ 
         $w_j * = \text{Gauss}(m_t, \text{noise}, M_t)$ 
    }
}

```

3.4 Resampling

Resampling concentrates the particle distribution around the most likely positions/greater weighted particle positions. The resampling ensures that particles representing less probable states are gradually removed, enhancing the accuracy of localization in subsequent iterations.

The algorithm for the resampling method we have adopted has been shown below where k denotes the current iteration:

```

 $S^k = \{x_j, w_j : j = 1 \dots M\}$ 
 $W = w_j : j = 1 \dots M$ 

Index = gen_real_random() * n;
beta = 0.0;
max_weight = max(w_j);
for j in range (1, M)
{
    beta += gen_real_random() * 2.0 * mw ;
    while ( beta > w[Index] )
    {
        beta -= w[Index] ;
        Index = any other Index;
    }
 $S^{k+1} = S^k(\text{Index})$ ;
}

```

Due to noise generated, the new particles will diverge slightly from the position of just the high weighted particles.

4. BOT PARTICLE SIMULATION

The results from our simulation of localizing a bot with a 1000 generated particles on the world map is shown in Fig.4.1

5. PARALLELIZATION STRATEGY

Monte Carlo has great scope for parallelization, given inherent data independence between particles. Most computation was done through loops, making parallelization straightforward. Some of our important considerations include:

OpenMP: Reduced forking and joining where possible, by forking once and parallelizing enclosed loops since intermediate serial sections are small and computationally negligible.

OpenACC: Provision of data clauses to prevent GPU from doing expensive *copyins* and *copyouts* by default.

6. RESULTS

- The time taken for localization increases with increase in number of particles(problem size), the increase for all programs- serial, OpenMP, and OpenACC being nearly linear. OpenACC is observed to be much faster when compared to OpenMP, as plotted in Fig.6.1

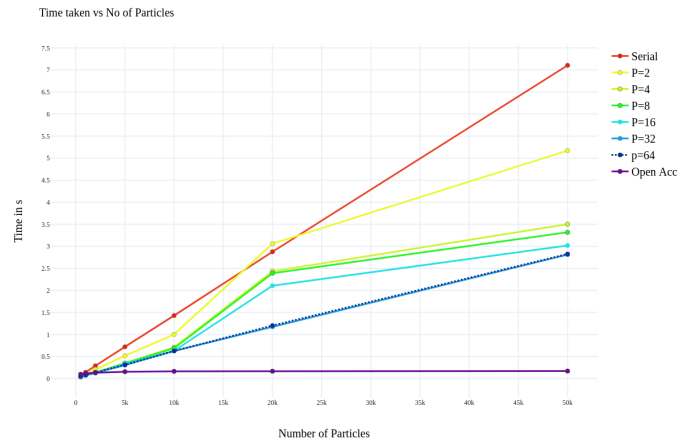


Fig.6.1 Time taken Vs Number of Particles

- The time taken decreases with an increase in the number of threads launched in OpenMP, for a given number of particles. There is an optimal number of threads for a given problem size and the time taken saturates for much larger number of threads as shown in Fig.6.2

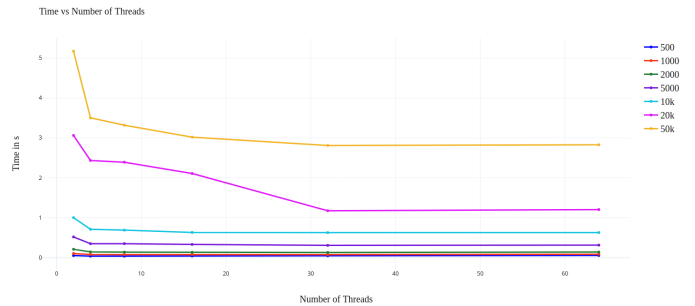


Fig.6.2 Time taken Vs Number of Threads

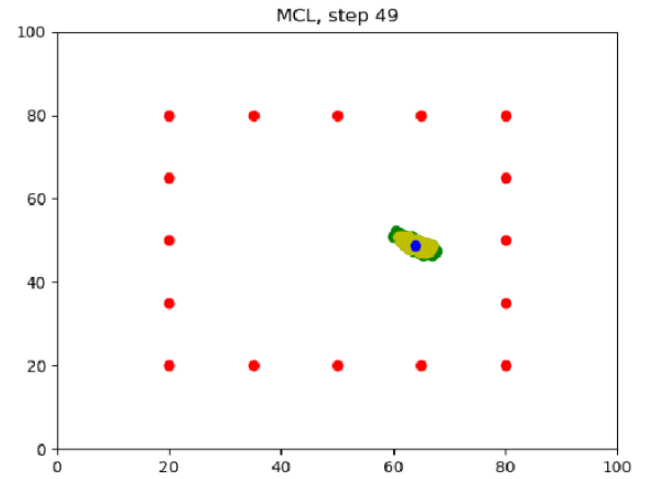
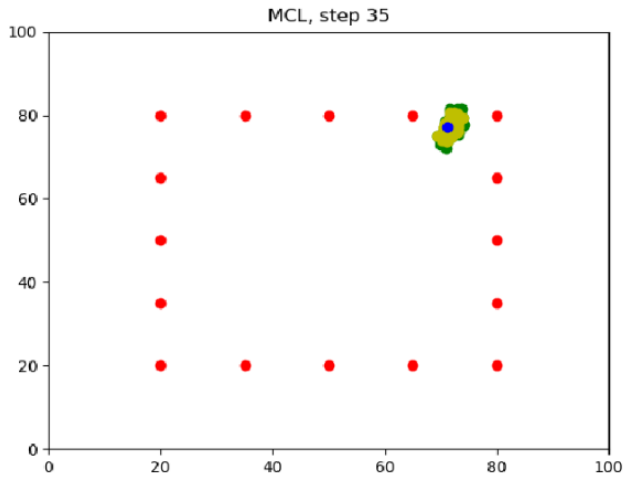
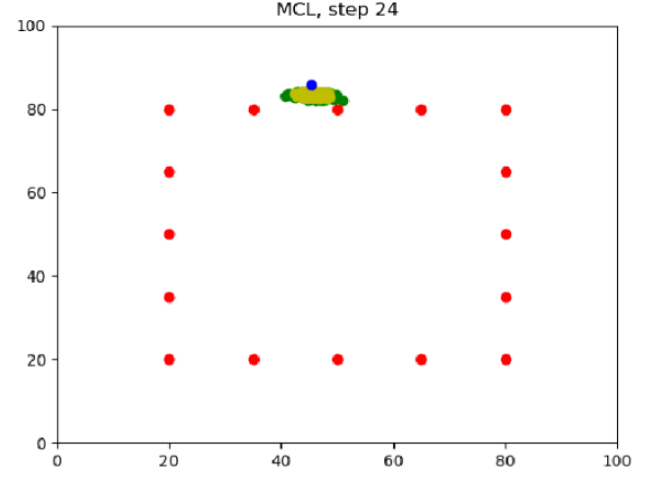
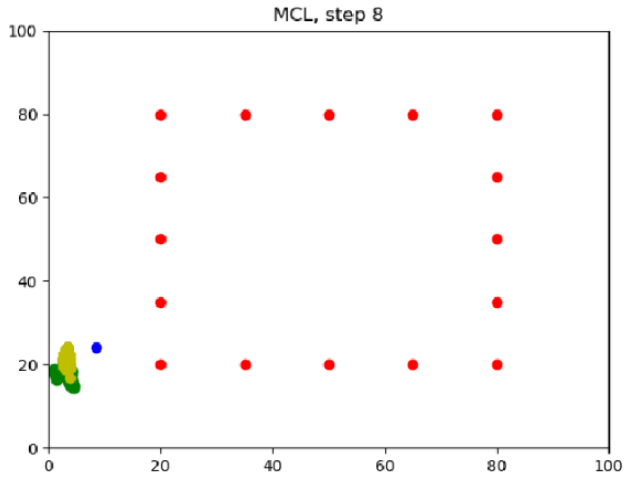
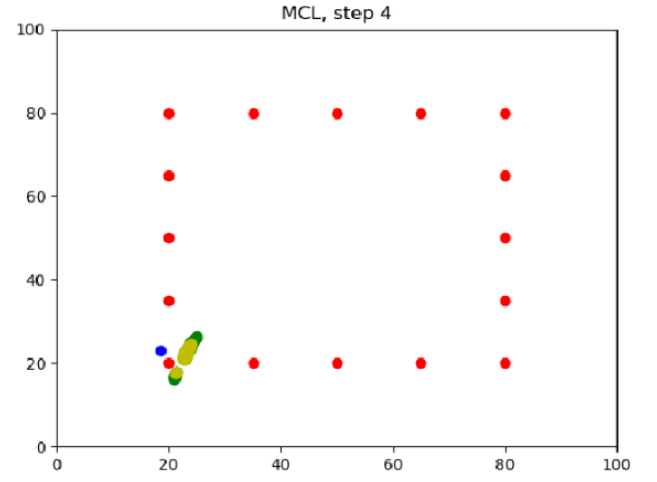
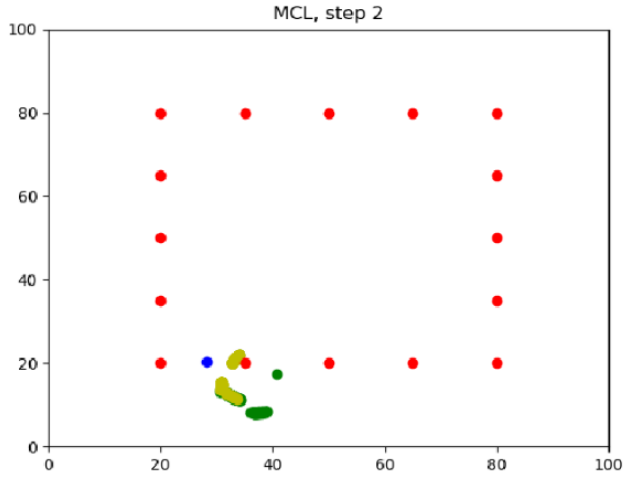
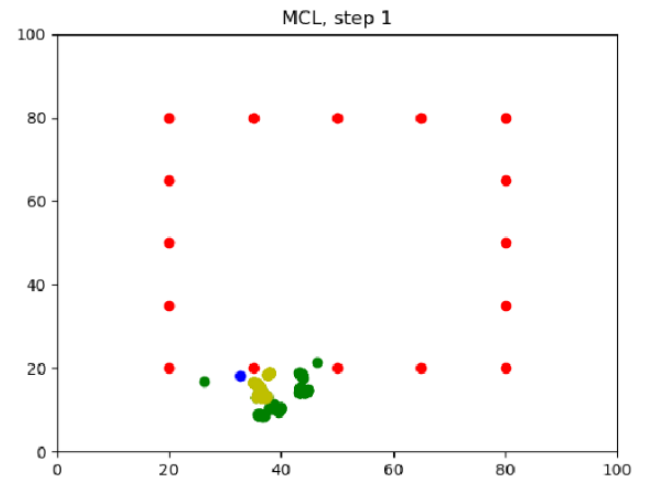
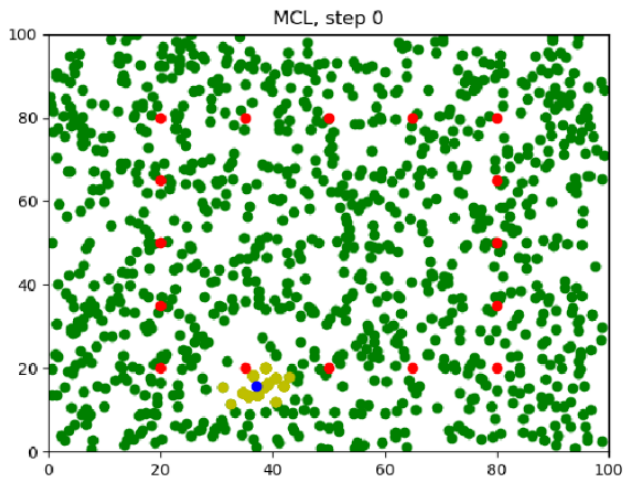


Fig.4.1 Simulation of localization at steps 0, 2, 4, 8, 16, 24, 35, 49 where blue, green, yellow represent the bot, the previous and the current iteration particles respectively

- The speedup (i.e. serial time taken/parallel time taken) increases up till a certain number of threads and then decreases (for a given problem size). The optimum number of threads for the best speedups increase with the problem size, as shown in **Fig.6.3**

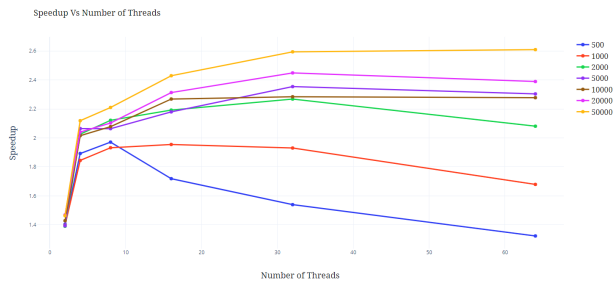


Fig.6.3 Speedup Vs Number of Threads- OpenMP

- Speedup varies for increase in small problem sizes (for a particular number of threads) in OpenMP, after which we observe a gradual linear increase in speedup for increasing problem sizes. Deviation from linearity at smaller number of particles may have occurred due to overheads associated with parallelization that were not compensated for smaller parallelizing sections, and Amdahl's Law. The plot is shown in **Fig.6.4**

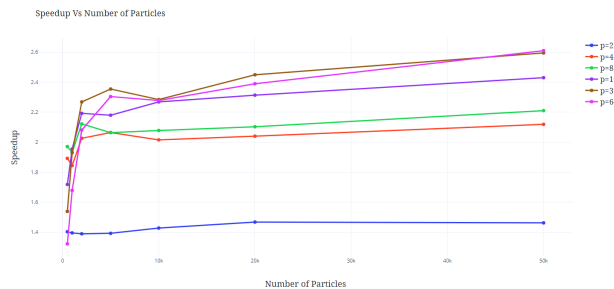


Fig.6.4 Speedup Vs Number of Particles - OpenMP

- It is evident from **Fig.6.5** that for larger problem sizes, OpenACC gives phenomenal speedups. At lower problem sizes, there are many overheads and communication expenses that do not justify the parallelization. The speedup attained increases linearly with problem size. For our smaller test windows, the largest speedup was observed to be 41, for the largest number of particles considered = 50k.

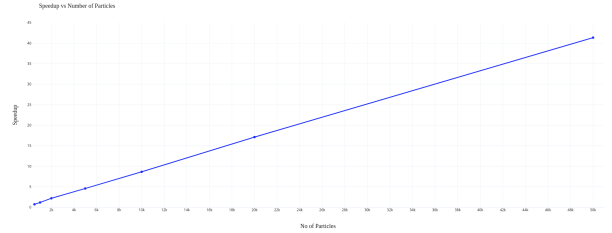


Fig.6.5 Speedup Vs Number of Particles - OpenACC

7. CONCLUSION

From the plots above, it is evident that we have obtained results closely aligning with the expected relationships between the time taken to localize and various parameters. This demonstrates the effectiveness of Monte Carlo methods in leveraging data parallelization. Our usage of parallel paradigms like OpenMP and OpenACC provided good speedups, particularly for an optimal number of threads for OpenMP, and increased problem sizes for both OpenACC and OpenMP.

ACKNOWLEDGMENT

Thanks goes to Prof. Kameswararao Anupindi for his guidance and support.

REFERENCES

- [1] Wikipedia, *Monte Carlo Localization*
- [2] Ioannis M. Rekleitis. "A Particle Filter Tutorial for Mobile Robot Localization.", Centre for Intelligent Machines, McGill University, Tech. Rep. TR-CIM-04-02 (2004).
- [3] Rémy Guyonneau, *Monte Carlo Localization for mobile robotics* - MCL github repo
- [4] Naoki Akai. *Reliable Monte Carlo Localization for Mobile Robots.*, Graduate School of Engineering, Nagoya University, Furo, Chikusa, Nagoya, Japan
- [5] Frank Dellaert et al, *Monte Carlo Localization for Mobile Robots*, 1999, Computer Science Department, Carnegie Mellon University, Pittsburgh PA 15213